Advanced Data Management

Mock Exam v2 — Chapters 1–6

Cycle Ingénieur · S3 · Fall 2026 · same format as the midterm

Duration:	2 hours (time yourself)	**Total:**	100 points
Allowed:	one A4 sheet of notes + formula sheet	**Name:**	_____

Instructions.

- Answer in **English**. Short, simple sentences are perfect: we grade the **technical content**, not the language.
- **Show your reasoning.** A wrong final number with a correct method earns most of the points; a correct number without a method earns little.
- Every situation uses the course database `adm_lab` (`customers` 10,000 rows / 73 pages, `products` 200 / 2, `orders` 500,000 / 3,185 pages), PostgreSQL 16, 8 KB pages. When a situation changes a size, it says so.
- Do it **before Monday 26 October**: the solution key is published on Canvas that day.
- Suggested timing: Part A 25 min · Part B 35 min · Part C 35 min · Part D 20 min · check 5 min.

Part	Type	What you do	Points
A	Quick checks	8 multiple-choice questions, one-line justification	24
B	Problem situations	3 decisions, each built on a short calculation	30
C	Read a real plan	annotate real `EXPLAIN` outputs from `adm_lab`	26
D	Discussion	**choose 2 of 3** tickets, half a page each	20
		Total	**100**

Part 1. Quick checks — circle one and justify in one line (24 pts)

3 points each: 1 for the letter, 2 for the justification (a number, a formula or one sentence).

1. Which component decides whether `orders` is read by a Seq Scan or by an Index Scan? **[3 pts]**
   (a) the parser (b) the planner (c) the executor (d) the buffer manager

2. You delete half of the rows of `orders` (random rows) and run a normal `VACUUM`. How many pages does the table file have afterwards? **[3 pts]**
   (a) about 1,592 (b) about 3,185 (c) about 6,370 (d) 0

3. There is an index on `orders (cust_id, order_date)`. Which query can use it the best? **[3 pts]**
   (a) `WHERE order_date = '2025-03-01'`
   (b) `WHERE cust_id = 7 AND order_date > '2025-06-01'`
   (c) `WHERE cust_id + 0 = 7` (d) `ORDER BY order_date LIMIT 10`

4. The planner joins `customers` (73 pages) and `orders` (3,185 pages) with a hash join. Which table should be the *build* side, and why? **[3 pts]**
(a) `orders`, the larger one (b) `customers`, the smaller one (c) either, the cost is the same (d) both are hashed

5. A file of 6,000 pages is sorted with $M = 100$ buffer pages. Counting run generation as one pass, how many passes? **[3 pts]**
(a) 1 (b) 2 (c) 3 (d) 60

6. `amount` is uniform between 5 and 105, and `orders` has 200 distinct `product_id`. How many rows does the planner estimate for `WHERE amount > 95 AND product_id = 7`? **[3 pts]**
(a) about 250 (b) about 2,500 (c) about 50,000 (d) about 52,500

7. A plan shows `Buffers: shared hit=40 read=3145`. What does `read=3145` mean? **[3 pts]**
(a) 3,145 pages were read from the disk (b) 3,145 pages were not in shared buffers; some may come from the OS cache
(c) the hit ratio is 40 % (d) 3,145 rows were read

8. The buffer manager uses **steal** + **no-force**. What must the log contain to recover after a crash? **[3 pts]**
(a) undo only (b) redo only (c) undo and redo (d) nothing, pages are on disk

Part 2. Problem situations — calculate, then decide (30 pts)

Use the formula sheet. Each situation ends with a decision: answer it in one or two sentences that use your numbers.

B1 — "Doubling work_mem will halve the sort time" (10 pts)

Every night, a report sorts the 1,200 pages of `orders_archive` by date, with $M = 25$ buffer pages for the sort. The DBA claims: *"If we double the memory to 50 pages, the sort will take half the time."*

(a) Today ($M = 25$): number of sorted runs, number of passes, total I/O (read + write in every pass). **[4 pts]**

(b) With $M = 50$: same three numbers. **[3 pts]**

(c) Is the DBA right? How much memory would make the sort a single pass? **[3 pts]**

Start with: "With 25 pages, the sort makes ... passes, so ..."

B2 — "GET /orders/{id} must answer in less than 1 ms" (10 pts)

The `orders` table of a large client has **20,000,000** rows. Its B$^+$-tree on `id` holds about **250** entries per page (leaves and internal pages). The API looks up one order by `id`. A random page read costs **100** μ**s** on SSD and **10 ms** on HDD; a page found in shared buffers costs **0.1** μ**s**.

(a) How many levels does the tree have? How many page reads for one lookup, heap page included? **[4 pts]**

(b) Worst case (nothing in memory): latency on SSD, and on HDD. Is the 1 ms target met? **[3 pts]**

(c) In practice the root and the second level are always in shared buffers. New latency on SSD? What should the team buy or keep? **[3 pts]**

Start with: "The tree has ... levels because ..."

B3 — "Hash joins are buggy, I switched them off" (10 pts)

A colleague runs a report with `SET enable_hashjoin = off`. It joins `customers_big` (R = **80** pages) and `orders_2025` (S = **3,200** pages) on `cust_id`, without a useful index. The query gets $M = 10$ buffer pages. The planner now uses a block nested loop with R as the outer relation.

(a) Cost of the block nested loop, in page I/O. **[3 pts]**

(b) Cost of a hash join with $M = 10$. Does R fit in memory? Which hash join variant applies? **[3 pts]**

(c) The colleague also raises the memory to $M = 100$ pages. Recompute both costs. **[2 pts]**

(d) What do you tell the colleague? **[2 pts]**

Start with: "The block nested loop reads S ... times, so ..."

Part 3. Read a real plan (26 pts)

Ticket REP-12 — "The December revenue report is slow after a restart"

The finance team runs this report. Below are three real executions on `adm_lab` (PostgreSQL 16.15, parallelism off): **run 1** just after a server restart, **run 2** immediately after, **run 3** after `SET work_mem = '64kB'`.

```
SELECT c.country, sum(o.amount) AS revenue
FROM orders o JOIN customers c ON c.id = o.cust_id
WHERE o.order_date >= '2025-12-01'
GROUP BY c.country ORDER BY revenue DESC;
```

```
-- run 1 (just after a restart)
Sort  (cost=10064.00..10064.01 rows=5) (actual time=82.857..82.860 rows=5 loops=1)
  Sort Key: (sum(o.amount)) DESC
  Sort Method: quicksort  Memory: 25kB
  Buffers: shared hit=3 read=3258
  ->  HashAggregate  (cost=10063.88..10063.94 rows=5) (actual rows=5 loops=1)
        Group Key: c.country
        ->  Hash Join  (cost=298.00..9846.94 rows=43388) (actual rows=42299 loops=1)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (cost=0.00..9435.00 rows=43388) (actual rows=42299 loops=1)
                    Filter: (order_date >= '2025-12-01'::date)
                    Rows Removed by Filter: 457701
                    Buffers: shared read=3185
              ->  Hash  (actual rows=10000 loops=1)
                    Buckets: 16384  Batches: 1  Memory Usage: 519kB
                    ->  Seq Scan on customers c  (cost=0.00..173.00 rows=10000) (actual rows=10000 loops=1)
                          Buffers: shared read=73
Execution Time: 82.963 ms
-- run 2: same plan, every "read" becomes "hit"   Buffers: shared hit=3258   Execution Time: 62.288 ms
-- run 3 (work_mem = 64kB): same plan, except
              ->  Hash  (actual rows=10000 loops=1)
                    Buckets: 4096  Batches: 8  Memory Usage: 80kB
                    Buffers: shared hit=73, temp written=28
  Buffers: shared hit=3258, temp read=183 written=183      Execution Time: 71.524 ms
```

1. Draw the plan as a tree (5 nodes). For each node, name the operator and say whether it must read all its input before producing its first row. **[5 pts]**

2. Is the estimate of the Seq Scan on `orders` good? Give the q-error and say where the estimate comes from. **[4 pts]**

3. Explain the `Buffers` lines of runs 1 and 2: what is 3,258, why does `read` become `hit`, and why is run 1 only about 1.3× slower and not 1,000×? **[6 pts]**

4. A colleague proposes `CREATE INDEX ON orders (order_date)` to speed up the report. In `pg_stats`, the `correlation` of `order_date` is -0.0015. Will the index help? Justify with numbers. **[6 pts]**

   Start with: "The query keeps ... % of the rows, and because the correlation is ..."

5. Run 3: what changed in the `Hash` node, what do `Batches: 8` and the `temp` lines mean, and what is your recommendation for `work_mem` for this report? **[5 pts]**

Part 4. Discussion — choose 2 of the 3 tickets (20 pts)

10 points each. Half a page. A good answer: ***claim*** $\rightarrow$ ***mechanism*** $\rightarrow$ ***number*** $\rightarrow$ ***consequence***. *Cross out the ticket you do not answer. Write on the answer sheets.*

1. **Ticket D1.** A server has 64 GB of RAM and runs only PostgreSQL. The DBA wants `shared_buffers = 52GB` (80 %) "because more cache is always better". Write your reply: what happens to the OS page cache, to sorts and hash joins, and to checkpoints? What would you measure before deciding? **[10 pts]**

   Start with: "I disagree, because PostgreSQL uses ..."

2. **Ticket D2.** A junior developer sets `enable_nestloop = off` and `enable_mergejoin = off` for the whole database, "because hash join is always the best". Give one situation where each disabled algorithm is clearly the best, and explain the consequences of the change. **[10 pts]**

   Start with: "Hash join is not always the best: for example ..."

3. **Ticket D3.** A manager asks: *"Our COMMIT takes 2 ms, but the transaction changed 300 pages. How can the data be safe if those pages are not written to disk at commit?"* Explain, then describe what happens after a power cut to a committed transaction and to an uncommitted one. **[10 pts]**

   Start with: "At COMMIT, PostgreSQL only writes ..."

Formula sheet

Topic	Formula
Pages of a table	pages = ⌈ rows / rows per page ⌉ · page = 8 KB
B$^+$-tree	levels = $\lceil \log_F(\text{entries}) \rceil$ (F = entries per page) · lookup = levels +1 heap page
External merge sort (B pages, M buffers)	runs = $\lceil B/M \rceil$ · merge passes = $\lceil \log_{M-1}(\text{runs}) \rceil$ · total I/O = $2B \times (1 +$ merge passes)
Block nested loop (R outer)	$B_R + \lceil B_R/(M-2) \rceil \times B_S$
Index nested loop	$B_R + \lvert R \rvert \times$ (index pages + matching heap pages)
Sort-merge join	$\text{sort}(R) + \text{sort}(S) + B_R + B_S$
Hash join	build fits in memory: $B_R + B_S$ · otherwise (Grace): $3(B_R + B_S)$
Selectivity	= value: $1/n_{\text{distinct}}$ or its MCV frequency · range: fraction of histogram buckets · AND: $s_1 s_2$ · OR: $s_1 + s_2 - s_1 s_2$ · join on key: $\lvert R \rvert \lvert S \rvert / \max(n_R, n_S)$
q-error	$\max(\text{est}/\text{actual},\ \text{actual}/\text{est})$
PostgreSQL cost model	seq page 1 · random page 4 · per row 0.01 · per operator 0.0025
Effective access time	EAT $= h \cdot t_{\text{mem}} + (1-h) \cdot t_{\text{disk}}$ · hit ratio h = hit / (hit + read)
Buffer pool (PostgreSQL 16)	clock-sweep, `usage_count` 0–5 · ring of 256 KB for a seq scan of a table > `shared_buffers`/4
Orders of magnitude	RAM 0.1 μs · SSD random read 100 μs · HDD random read 10 ms · sequential read 500 MB/s